Caminos más cortos desde una fuente

De la búsqueda en amplitud a Bellman-Ford-Moore

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre de 2026

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Objetivos de la sesión

Al terminar la clase, usted podrá

- Calcular con la búsqueda en amplitud la distancia en aristas desde una fuente y reconstruir el camino con π.
- Definir $w(p)$ y $\delta(u, v)$, y decidir cuándo $\delta(s, v)$ vale ∞ o $-\infty$.
- Aplicar la relajación y usar sus propiedades para justificar un algoritmo.

Y también

- Ejecutar a mano e implementar Bellman-Ford, con parada temprana y con detección de ciclos negativos, en $O(VE)$.
- Construir la variante de Moore a partir de la versión por rondas y seguirla en una tabla.

La pregunta de hoy

Lo que ya se tiene

La búsqueda en amplitud recorre un grafo por niveles y deja en cada vértice cuántas aristas lo separan de la fuente.

La pregunta

Si cada arista tiene un costo, como el tiempo de un vuelo o el precio de un peaje, ¿sigue siendo cierto que el camino con menos aristas es el más barato? Y si algunos costos son negativos, ¿existe siempre un camino más barato?

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Lo que deja la amplitud en cada vértice

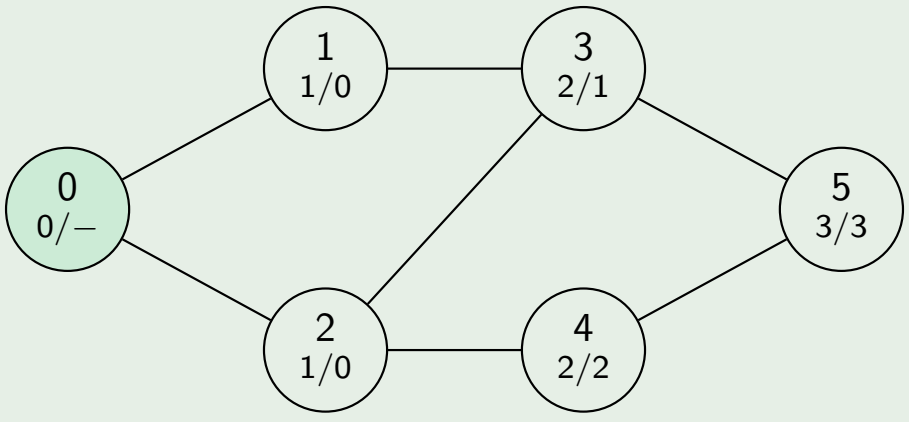
CLRS, Sección 22.2

$BFS(G, s)$ deja dos valores en cada vértice v:

- $v.d$: el número de aristas con que se llegó a v desde s; ∞ si nunca se llegó.
- $v.\pi$: el vértice desde el que se descubrió v; *NIL* para s y para los que no se alcanzan.

Lo que deja la amplitud en cada vértice

Desde 0, con las listas en orden creciente



Debajo de cada vértice va d/π. La cola saca $0, 1, 2, 3, 4, 5$, en ese orden: primero todo lo que está a una arista, luego lo que está a dos.

El código

```
def bfs(G, s):
    # d[v]: aristas de s a v; pi[v]: el anterior en el camino.
    n = len(G)
    d = [-1] * n
    pi = [-1] * n
    d[s] = 0
    cola = deque()
    cola.append(s)
    while len(cola) > 0:
        u = cola.popleft()
        for v in G[u]:
            if d[v] == -1:
                d[v] = d[u] + 1
                pi[v] = u
                cola.append(v)
    return (d, pi)
```

El código

Dos detalles

`d[v] == -1` hace las veces de ∞ y de color blanco a la vez. Cada vértice entra a la cola una sola vez, así que el costo es $\Theta(V + E)$.

El teorema y la reconstrucción del camino

Teorema (CLRS, Teorema 22.5)

Sea $G = (V, E)$ un grafo, dirigido o no, y $s \in V$. Al terminar BFS(G, s), $v.d = \delta(s, v)$ para todo $v \in V$, donde $\delta(s, v)$ es el menor número de aristas de un camino de s a v. Además, para todo $v \neq s$ alcanzable, uno de los caminos más cortos de s a v es un camino más corto de s a $v.\pi$ seguido de la arista $(v.\pi, v)$.

En el grafo anterior

$5.\pi = 3$, $3.\pi = 1$, $1.\pi = 0$: el camino es $0, 1, 3, 5$, con $5.d = 3$ aristas. No hay uno de dos: 5 no es vecino de 1 ni de 2.

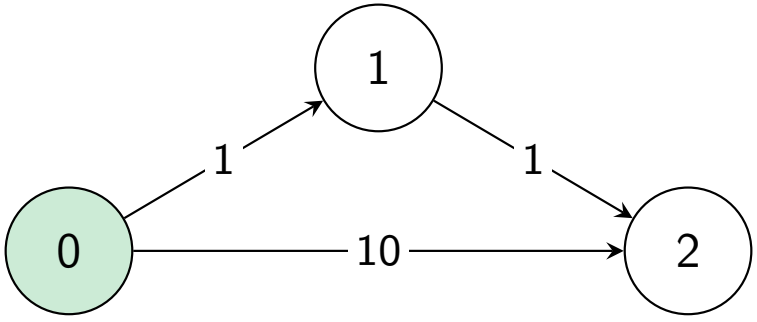
El teorema y la reconstrucción del camino

```python
def camino(pi, s, v):
    # Sube por pi desde v y voltea la lista; [] si no se llega.
    ruta = []
    x = v
    while x != -1:
        ruta.append(x)
        x = pi[x]
    ruta.reverse()
    if ruta[0] != s:
        ruta = []
    return ruta
```

Lo que hace

Es *PRINT-PATH* de CLRS sin recursión: sube por π desde v hasta *NIL* y voltea la lista. Si lo que queda arriba no es s, v no era alcanzable.

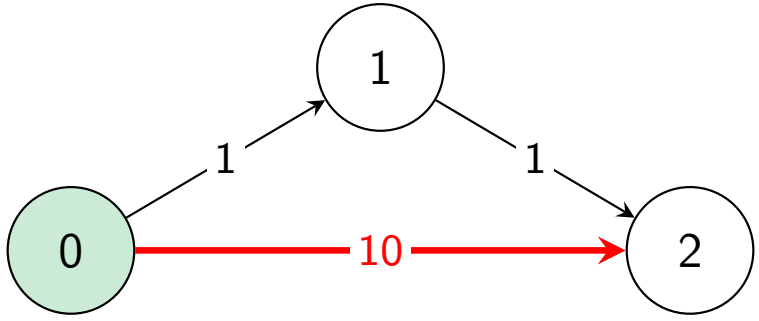
¿Y si las aristas tienen peso?



Pregunta

Ejecute la amplitud desde 0 con la lista de 0 en el orden $2, 1$. ¿Qué camino a 2 deja en π? ¿Es el más barato?

¿Y si las aristas tienen peso?



Respuesta

La amplitud descubre 2 desde 0 en el primer nivel y ahí se queda: $2.\pi = 0$ y el camino cuesta 10. El camino $0, 1, 2$ tiene una arista más y cuesta 2. Con el otro orden de la lista pasa lo mismo.

Lo que falla

La amplitud fija la distancia de un vértice la primera vez que lo ve. Con pesos, un camino que se descubre después puede ser más barato: hace falta poder *corregir* una distancia ya asignada.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Peso de un camino y distancia

Definición (CLRS, Capítulo 24)

Sea $G = (V, E)$ dirigido con una función de peso $w : E \to \mathbb{R}$. El peso de un camino $p = \langle v_0, v_1, \ldots, v_k \rangle$ es

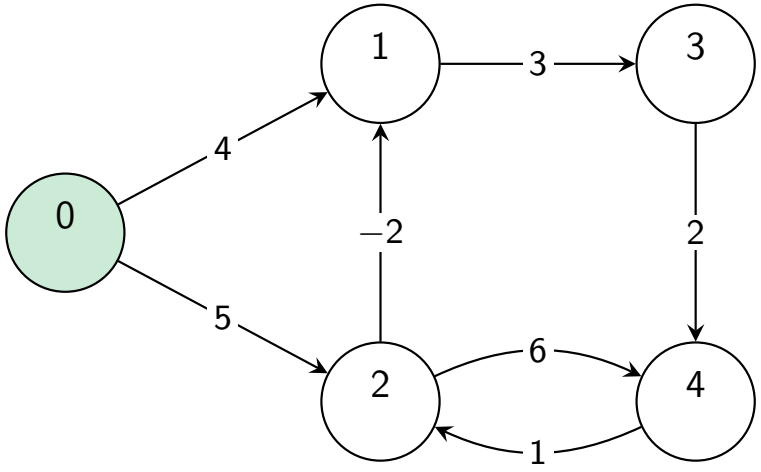
$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

El peso del camino más corto de u a v es

$$\delta(u, v) = \begin{cases} \text{mín}\{\, w(p) : u \overset{p}{\rightsquigarrow} v \,\} & \text{si hay camino de } u \text{ a } v, \\ \infty & \text{si no lo hay.} \end{cases}$$

Un camino más corto de u a v es cualquier p con $w(p) = \delta(u, v)$.

Peso de un camino y distancia

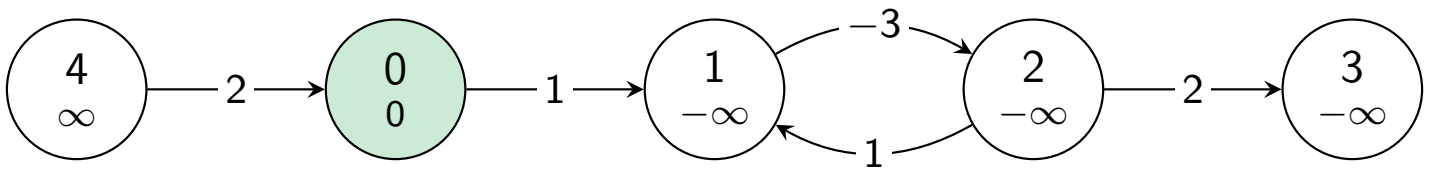


Ejemplo: de 0 a 4

- $\langle 0, 1, 3, 4 \rangle$: $4 + 3 + 2 = 9$, con tres aristas.
- $\langle 0, 2, 4 \rangle$: $5 + 6 = 11$, con dos aristas.
- $\langle 0, 2, 1, 3, 4 \rangle$: $5 - 2 + 3 + 2 = 8$, con cuatro.

$\delta(0, 4) = 8$ y lo da el camino con más aristas. Este grafo es el de toda la sesión.

Pesos negativos y ciclos negativos



Desde la fuente 0; debajo de cada vértice, $\delta(0, v)$

- El ciclo $\langle 1, 2, 1 \rangle$ pesa $-3 + 1 = -2$. Cada vuelta baja el costo en 2: no hay mínimo y $\delta(0, 1) = \delta(0, 2) = -\infty$.
- 3 cuelga del ciclo, así que también $\delta(0, 3) = -\infty$.
- A 4 no se llega desde 0: $\delta(0, 4) = \infty$, aunque 4 sí llegue a 0.

Pesos negativos y ciclos negativos

Lo que dice CLRS (Capítulo 24, pesos negativos)

Las aristas negativas no son el problema. El problema es un ciclo de peso negativo *alcanzable desde la fuente*: si existe, cualquier vértice al que se llegue pasando por él tiene $\delta(s, v) = -\infty$. Un ciclo negativo al que no se llega desde s no afecta ninguna distancia desde s.

En el grafo de la sesión

La única arista negativa es $2 \to 1$, de peso -2, y el único ciclo es $\langle 2, 1, 3, 4, 2 \rangle$, de peso $-2 + 3 + 2 + 1 = 4$. No hay ciclo negativo y todas las distancias desde 0 son finitas.

Subestructura óptima

Lema (CLRS, Lema 24.1)

Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ *un camino más corto de* v_0 *a* v_k *y, para* $0 \leq i \leq j \leq k$, *sea* $p_{ij} = \langle v_i, \ldots, v_j \rangle$ *el subcamino de* p *entre* v_i *y* v_j. *Entonces* p_{ij} *es un camino más corto de* v_i *a* v_j.

Ejemplo

$\langle 0, 2, 1, 3, 4 \rangle$ es más corto de 0 a 4. Su tramo $\langle 2, 1, 3 \rangle$ pesa 1, y no hay otro camino de 2 a 3: es más corto. Su tramo $\langle 0, 2, 1 \rangle$ pesa 3, menos que la arista directa $0 \to 1$, de peso 4.

Subestructura óptima

Demostración

Se procede por contradicción.

Desarrollo

Se parte p en tres tramos: $v_0 \overset{p_{0i}}{\rightsquigarrow} v_i \overset{p_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$, así que $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Si existiera un camino p'_{ij} de v_i a v_j con $w(p'_{ij}) < w(p_{ij})$, el camino $v_0 \overset{p_{0i}}{\rightsquigarrow} v_i \overset{p'_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$ pesaría $w(p_{0i}) + w(p'_{ij}) + w(p_{jk}) < w(p)$, y p no sería más corto.

Conclusión

Por lo tanto, se puede concluir que todo subcamino de un camino más corto es un camino más corto entre sus extremos.

Un camino más corto no tiene ciclos

Pregunta

Si un camino más corto repitiera un vértice, ¿qué peso podría tener el ciclo entre las dos visitas?

Los tres casos (CLRS, Capítulo 24, ciclos)

- **Negativo**: entonces $\delta(s, v) = -\infty$ y no hay camino más corto.
- **Positivo**: quitarlo da un camino más barato, así que el camino no era más corto.
- **Cero**: quitarlo da otro camino con el mismo peso.

Un camino más corto no tiene ciclos

Consecuencia

Sin ciclos negativos alcanzables siempre hay un camino más corto *simple*, con a lo sumo $|V| - 1$ aristas. En el grafo de la sesión, $\langle 0, 2, 1, 3, 4, 2, 1, 3, 4 \rangle$ pesa 12: le sobra una vuelta de peso 4.

La representación con π

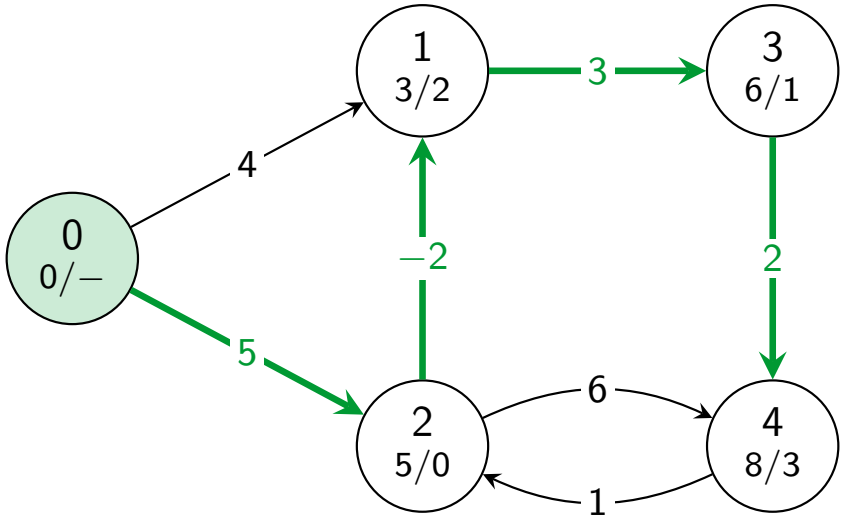
CLRS, Capítulo 24, representación de los caminos

Como en la amplitud, cada vértice guarda su predecesor $v.\pi$. Al terminar, el subgrafo G_π de las aristas $(v.\pi, v)$ es un *árbol de caminos más cortos* con raíz s: el camino del árbol de s a v es un camino más corto en G.

Pregunta

En el grafo de la sesión, ¿qué aristas forman el árbol? Pista: cada vértice distinto de 0 tiene exactamente una arista que entra en él desde su π.

La representación con π



En el grafo de la sesión

Debajo de cada vértice va δ/π. Desde 4 se sube a 3, 1, 2 y 0: el camino es $\langle 0, 2, 1, 3, 4\rangle$, y la misma función `camino` lo reconstruye.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Inicializar y relajar

CLRS, Capítulo 24, relajación

INITIALIZE-SINGLE-SOURCE(G, s)
1. para cada $v \in G.V$: $v.d = \infty$, $v.\pi = NIL$
2. $s.d = 0$

RELAX(u, v, w)
1. si $v.d > u.d + w(u, v)$
2. $\quad v.d = u.d + w(u, v)$
3. $\quad v.\pi = u$

Inicializar y relajar

Qué es $v.d$

Una *estimación*: el mejor camino de s a v conocido hasta ahora. Relajar (u, v) es preguntar si pasar por u la mejora; es la corrección que la amplitud no hacía.

Inicializar y relajar

```
INF = float('inf')

def inicializar(n, s):
    # INITIALIZE-SINGLE-SOURCE: infinito salvo en la fuente.
    d = [INF] * n
    pi = [-1] * n
    d[s] = 0
    return (d, pi)

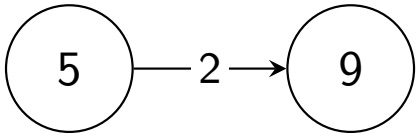
def relajar(u, v, w, d, pi):
    # RELAX; devuelve True si d[v] cambio.
    cambio = False
    if d[u] + w < d[v]:
        d[v] = d[u] + w
        pi[v] = u
        cambio = True
    return cambio
```

Inicializar y relajar

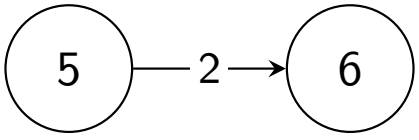
Una diferencia con CLRS

`relajar` devuelve si cambió `d[v]`; la parada temprana y la variante de Moore usan ese valor.

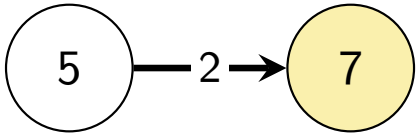
Relajar una arista



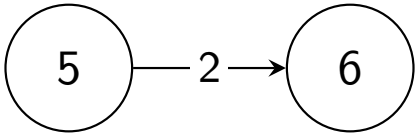
antes



antes



después: $9 > 5 + 2$



después: $6 \leq 5 + 2$, no cambia

CLRS, Figura 24.3

Dentro de cada vértice va su estimación d; a la izquierda u, a la derecha v. En el primer caso la arista mejora a v y $v.\pi$ pasa a ser u. En el segundo, v ya tenía algo mejor y nada cambia.

Las propiedades de la relajación

CLRS, Sección 24.5

Todas suponen que d y π se inicializaron con *INITIALIZE-SINGLE-SOURCE*(G, s) y que después solo cambian con *RELAX*, en cualquier orden y cualquier número de veces.

Desigualdad triangular (Lema 24.10)

Para toda arista (u, v): $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Ejemplo

En el grafo de la sesión, $\delta(0, 4) = 8 \leq \delta(0, 2) + w(2, 4) = 5 + 6 = 11$. La igualdad se da en las aristas del árbol: $\delta(0, 4) = \delta(0, 3) + w(3, 4) = 6 + 2$.

Las propiedades de la relajación

Cota superior (Lema 24.11)

Siempre $v.d \geq \delta(s, v)$, y cuando $v.d$ alcanza $\delta(s, v)$ ya no cambia.

Ejemplo

En la traza que viene, $3.d$ pasa por ∞, luego 7, luego $6 = \delta(0, 3)$, y ahí se queda. Nunca baja de 6.

Las propiedades de la relajación

Ausencia de camino (Corolario 24.12)

Si no hay camino de s a v, siempre $v.d = \infty$.

Ejemplo

En el grafo del ciclo negativo, $4.d$ se queda en ∞: ninguna arista entra a 4, y *RELAX* solo baja $v.d$ a partir de un $u.d$ finito.

Convergencia (Lema 24.14)

Si $s \rightsquigarrow u \to v$ es un camino más corto y $u.d = \delta(s, u)$ antes de relajar (u, v), después $v.d = \delta(s, v)$.

Ejemplo

Cuando $1.d$ ya vale $3 = \delta(0, 1)$, relajar $(1, 3)$ deja $3.d = 3 + 3 = 6 = \delta(0, 3)$.

Las propiedades de la relajación

Relajación de caminos (Lema 24.15)

Si $p = \langle v_0, v_1, \ldots, v_k \rangle$ es un camino más corto de $s = v_0$ a v_k y se relajan sus aristas en el orden $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, aunque entre ellas se relajen otras, al final $v_k.d = \delta(s, v_k)$.

Ejemplo

Para $\langle 0, 2, 1, 3, 4 \rangle$ basta que $(0, 2)$ se relaje antes que $(2, 1)$, esta antes que $(1, 3)$ y esta antes que $(3, 4)$. Las demás aristas pueden relajarse en medio, cuantas veces sea.

Pregunta

Si no se conoce el camino más corto, ¿cómo garantizar que sus aristas se relajen en orden? Ahí empieza Bellman-Ford.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

La idea: rondas completas

Por tanteo

El camino más corto a 4 tiene cuatro aristas. Si en cada ronda se relajan *todas* las aristas del grafo, en la primera se relaja $(0, 2)$, en la segunda $(2, 1)$, en la tercera $(1, 3)$ y en la cuarta $(3, 4)$: el orden del camino queda garantizado, sin conocerlo.

¿Cuántas rondas?

Tantas como aristas tenga el camino más corto más largo. Sin ciclos negativos hay uno simple, con a lo sumo $|V| - 1$ aristas. Entonces $|V| - 1$ rondas alcanzan.

La idea: rondas completas

BELLMAN-FORD(G, w, s), CLRS, Sección 24.1

1. *INITIALIZE-SINGLE-SOURCE*(G, s)
2. para $i = 1$ hasta $|G.V| - 1$
3. para cada arista $(u, v) \in G.E$
4. *RELAX*(u, v, w)
5. para cada arista $(u, v) \in G.E$
6. si $v.d > u.d + w(u, v)$
7. devolver *FALSE*
8. devolver *TRUE*

Las dos partes

Las líneas 2 a 4 calculan las distancias; las líneas 5 a 7 buscan una arista que todavía se pueda relajar.

La versión ingenua

```
def hay_arista_tensa(G, d):
    # Una arista tensa despues de las rondas delata un ciclo
    # negativo alcanzable desde la fuente.
    tensa = False
    for u in range(len(G)):
        for (v, w) in G[u]:
            if d[u] + w < d[v]:
                tensa = True
    return tensa
```

Arista tensa

Una arista (u, v) con $v.d > u.d + w(u, v)$: relajarla todavía cambia algo. La revisión se escribe aparte porque las tres versiones la usan.

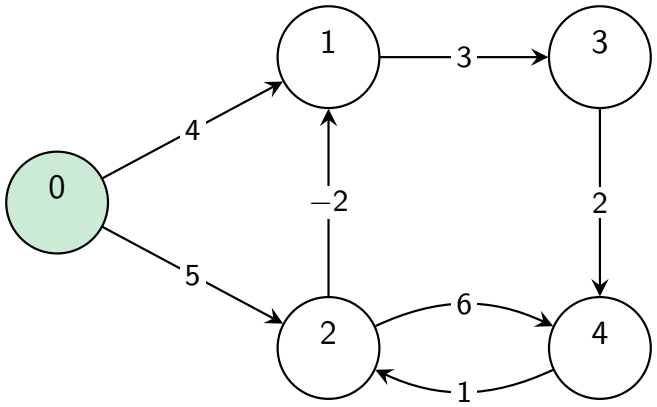
La versión ingenua

```python
def bellman_ford_ingenuo(G, s):
    # |V|-1 rondas completas, pase lo que pase.
    n = len(G)
    d, pi = inicializar(n, s)
    ronda = 1
    while ronda <= n - 1:
        for u in range(n):
            for (v, w) in G[u]:
                relajar(u, v, w, d, pi)
        ronda = ronda + 1
    sin_ciclo = not hay_arista_tensa(G, d)
    return (d, pi, sin_ciclo)
```

Lo que cuesta

Inicializar cuesta $\Theta(V)$. Cada ronda recorre las listas completas, $\Theta(V + E)$, y hay $|V| - 1$ rondas. La revisión final es una ronda más. Total: $\Theta(VE)$ cuando $E \geq V$, sin importar el grafo.

Traza: ronda 1



Las aristas se relajan en el orden de las listas: $u = 0, 1, \ldots, 4$ y, dentro de cada lista, en el orden en que aparecen. En rojo, lo que cambia.

Arista	0	1	2	3	4	¿Cambia?
inicio	0	∞	∞	∞	∞	
$(0,1)$, 4	0	4	∞	∞	∞	sí, $1.\pi = 0$
$(0,2)$, 5	0	4	5	∞	∞	sí, $2.\pi = 0$
$(1,3)$, 3	0	4	5	7	∞	sí, $3.\pi = 1$
$(2,1)$, -2	0	3	5	7	∞	sí, $1.\pi = 2$
$(2,4)$, 6	0	3	5	7	11	sí, $4.\pi = 2$
$(3,4)$, 2	0	3	5	7	9	sí, $4.\pi = 3$
$(4,2)$, 1	0	3	5	7	9	no: $5 \leq 9 + 1$

Traza: rondas 2 a 4

Pregunta

En la ronda 1, $3.d$ quedó en 7 porque $(1, 3)$ se relajó cuando $1.d$ todavía valía 4. ¿Qué arista lo arregla en la ronda 2, y qué arrastra ese cambio?

Ronda 2	0	1	2	3	4	¿Cambia?
$(0, 1)$, 4	0	3	5	7	9	no
$(0, 2)$, 5	0	3	5	7	9	no
$(1, 3)$, 3	0	3	5	6	9	sí: $7 > 3 + 3$
$(2, 1)$, -2	0	3	5	6	9	no
$(2, 4)$, 6	0	3	5	6	9	no
$(3, 4)$, 2	0	3	5	6	8	sí: $9 > 6 + 2$
$(4, 2)$, 1	0	3	5	6	8	no

Traza: rondas 2 a 4

Rondas 3 y 4

Ninguna arista cambia nada: $d = (0, 3, 5, 6, 8)$ y $\pi = (\mathit{NIL}, 2, 0, 1, 3)$, que son las δ y el árbol que ya se vieron. La revisión final no encuentra aristas tensas y la función devuelve `True`.

Lo que se pagó

La versión ingenua examinó $4 \times 7 = 28$ aristas en las rondas, más 7 en la revisión. Las rondas 3 y 4 no cambiaron nada: 14 exámenes perdidos.

Las propiedades, en la traza

$1.d = 3$ se fijó en la ronda 1, por convergencia desde 2. $(1, 3)$ se relajó con ese valor en la ronda 2 y $(3, 4)$ después, en la misma ronda: el camino $\langle 0, 2, 1, 3, 4 \rangle$ se relajó en orden y $4.d = 8$. Dos rondas bastaron porque el orden de las listas ayudó en la primera.

Por qué $|V| - 1$ rondas bastan

Lema (CLRS, Lema 24.2)

Sea $G = (V, E)$ dirigido con pesos w y fuente s, sin ciclos de peso negativo alcanzables desde s. Después de las $|V| - 1$ rondas de las líneas 2 a 4, $v.d = \delta(s, v)$ para todo v alcanzable desde s.

Demostración

Se procede de forma directa, con la propiedad de relajación de caminos.

Por qué $|V| - 1$ rondas bastan

Desarrollo

Sea v alcanzable. Como no hay ciclos negativos alcanzables, existe un camino más corto simple $p = \langle v_0, v_1, \ldots, v_k \rangle$ con $v_0 = s$, $v_k = v$ y $k \leq |V| - 1$. Cada ronda relaja todas las aristas, así que la ronda i relaja (v_{i-1}, v_i) para $i = 1, \ldots, k$. Las aristas de p quedan relajadas en el orden del camino, con otras relajaciones en medio.

Conclusión

Por lo tanto, por la propiedad de relajación de caminos, después de la ronda $k \leq |V| - 1$ se tiene $v.d = \delta(s, v)$, y por la cota superior ya no cambia en las rondas que siguen.

Correctitud de Bellman-Ford

Teorema (CLRS, Teorema 24.4)

Sea BELLMAN-FORD ejecutado sobre G con pesos w y fuente s. Si G no tiene ciclos negativos alcanzables desde s, devuelve TRUE, $v.d = \delta(s, v)$ para todo $v \in V$ y G_π es un árbol de caminos más cortos con raíz s. Si G tiene un ciclo negativo alcanzable desde s, devuelve FALSE.

La primera parte, en una línea

Por el Lema 24.2 y la ausencia de camino, al terminar las rondas $v.d = \delta(s, v)$ para todo v. Entonces, por la desigualdad triangular, $v.d = \delta(s, v) \leq \delta(s, u) + w(u, v) = u.d + w(u, v)$ en cada arista, ninguna está tensa y se devuelve *TRUE*.

Correctitud de Bellman-Ford

Demostración de la segunda parte

Se procede por contradicción: se supone que hay un ciclo negativo $c = \langle v_0, v_1, \ldots, v_k \rangle$, con $v_0 = v_k$, alcanzable desde s, y que el algoritmo devuelve *TRUE*.

Desarrollo

Si devuelve *TRUE*, ninguna arista de c está tensa: $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ para $i = 1, \ldots, k$. Sumando las k desigualdades, $\sum_{i=1}^{k} v_i.d \leq \sum_{i=1}^{k} v_{i-1}.d + \sum_{i=1}^{k} w(v_{i-1}, v_i)$. Como $v_0 = v_k$, las dos sumas de d recorren los mismos vértices y son iguales; son finitas porque cada v_i es alcanzable. Restando, $0 \leq w(c)$.

Correctitud de Bellman-Ford

Conclusión

Eso contradice $w(c) < 0$. Por lo tanto, se puede concluir que con un ciclo negativo alcanzable alguna arista queda tensa y el algoritmo devuelve *FALSE*.

Parada temprana

Pregunta

En la traza, las rondas 3 y 4 no cambiaron nada. Si una ronda completa no cambia ningún d, ¿puede cambiar algo la siguiente?

Respuesta

No. La ronda siguiente relaja las mismas aristas con los mismos d, y ninguna estaba tensa. Lo mismo vale para todas las que vienen después. Se puede parar en ese momento.

Parada temprana

```
def bellman_ford(G, s):
    # Si una ronda no cambia nada, las siguientes tampoco.
    n = len(G)
    d, pi = inicializar(n, s)
    ronda = 1
    hubo_cambio = True
    while ronda <= n - 1 and hubo_cambio:
        hubo_cambio = False
        for u in range(n):
            for (v, w) in G[u]:
                if relajar(u, v, w, d, pi):
                    hubo_cambio = True
        ronda = ronda + 1
    sin_ciclo = not hay_arista_tensa(G, d)
    return (d, pi, sin_ciclo)
```

Parada temprana

Lo que cambió

La bandera `hubo_cambio`, que entra en la condición del `while`. En el grafo de la sesión se hacen tres rondas, la tercera sin cambios: 21 exámenes de arista en lugar de 28. El peor caso sigue siendo $|V| - 1$ rondas y $\Theta(VE)$.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

¿Qué aristas valiá la pena relajar?

Pregunta

En la ronda 2 cambiaron $3.d$ y $4.d$. ¿Qué tenían en común las aristas que produjeron esos cambios?

Respuesta

$(1, 3)$ salió de 1, cuyo d había cambiado después de la última vez que se relajó $(1, 3)$. $(3, 4)$ salió de 3, que acababa de cambiar. Las aristas de 0 y de 2 se relajaron con los mismos d que ya habían usado: no podían dar nada nuevo.

La observación de Moore

Relajar (u, v) solo puede cambiar $v.d$ si $u.d$ cambió desde la última vez que se relajó (u, v). Basta con llevar la cuenta de los vértices cuyo d cambió y relajar solo las aristas que salen de ellos.

De las rondas a la cola, paso a paso

Paso 1: una cola de pendientes

En lugar de recorrer todos los u en cada ronda, se guardan en una cola los vértices cuyo d cambió. Al principio solo está s. Las rondas desaparecen: se saca un u y se relajan sus aristas.

```
cola = deque()
cola.append(s)
while len(cola) > 0:
    u = cola.popleft()
    for (v, w) in G[u]:
        if relajar(u, v, w, d, pi):
            cola.append(v)
```

De las rondas a la cola, paso a paso

Paso 2: nada repetido en la cola

Si v ya está esperando, cuando salga usará su d más reciente: meterlo otra vez no aporta nada. Un arreglo `en_cola` lo evita.

```
        if relajar(u, v, w, d, pi) and not en_cola[v]:
            cola.append(v)
            en_cola[v] = True
```

Paso 3: el ciclo negativo

Sin el contador de rondas, un ciclo negativo alcanzable hace que la cola nunca se vacíe: las distancias bajan sin fin. Se cuenta en `veces[v]` cuántas veces entra cada vértice; si alguno llega a $|V|$, hay ciclo negativo y se para.

```
            veces[v] = veces[v] + 1
            if veces[v] >= n:
                sin_ciclo = False
```

Bellman-Ford-Moore

```python
def bellman_ford_moore(G, s):
    # Solo se relajan las aristas de los vertices cuya d cambio.
    # veces[v]: cuantas veces ha entrado v a la cola.
    n = len(G)
    d, pi = inicializar(n, s)
    en_cola = [False] * n
    veces = [0] * n
    cola = deque()
    cola.append(s)
    en_cola[s] = True
    veces[s] = 1
    sin_ciclo = True
    while len(cola) > 0 and sin_ciclo:
        u = cola.popleft()
        en_cola[u] = False
        for (v, w) in G[u]:
            if relajar(u, v, w, d, pi) and not en_cola[v]:
                cola.append(v)
                en_cola[v] = True
                veces[v] = veces[v] + 1
                if veces[v] >= n:
                    sin_ciclo = False
    return (d, pi, sin_ciclo)
```

Bellman-Ford-Moore

Por qué basta contar hasta $|V|$

Como la cola es FIFO, los vértices salen por tandas: la tanda 0 es $\{s\}$ y la tanda k son los que entraron mientras salía la tanda $k-1$. Lo que la tanda k consigue es lo que conseguía la ronda k de Bellman-Ford, o algo mejor, y por `en_cola` un vértice entra a lo sumo una vez por tanda. Sin ciclos negativos alcanzables, a partir de la tanda $|V|-1$ ya no cambia nada, así que ningún vértice entra $|V|$ veces. Con un ciclo negativo la cola no se vacía y alguno llega a $|V|$.

Lo que cuesta

Cada vez que u sale de la cola se recorre su lista, $O(1+\text{grado}(u))$. Si cada vértice sale a lo sumo $|V|-1$ veces, el total es $O(VE)$ en el peor caso, como Bellman-Ford. La ganancia está en que solo se examinan las aristas que pueden cambiar algo.

Traza de Moore

Sale	Cambian	d					Cola	veces				
		0	1	2	3	4		0	1	2	3	4
		0	∞	∞	∞	∞	0	1	0	0	0	0
0	1, 2	0	4	5	∞	∞	1, 2	1	1	1	0	0
1	3	0	4	5	7	∞	2, 3	1	1	1	1	0
2	1, 4	0	3	5	7	11	3, 1, 4	1	2	1	1	1
3	4	0	3	5	7	9	1, 4	1	2	1	1	1
1	3	0	3	5	6	9	4, 3	1	2	1	2	1
4		0	3	5	6	9	3	1	2	1	2	1
3	4	0	3	5	6	8	4	1	2	1	2	2
4		0	3	5	6	8	vacía	1	2	1	2	2

Pregunta

Cuando sale 3 en la cuarta fila, 4.d baja de 11 a 9, pero 4 no se agrega a la cola. ¿Por qué?

Traza de Moore

Respuesta

4 ya estaba esperando: entró cuando salió 2. Cuando le toque salir usará $4.d = 9$, el valor más reciente, así que no hace falta otra copia. Eso es lo que evita `en_cola`.

Lo que se examinó

Ocho salidas de la cola y $2 + 1 + 2 + 1 + 1 + 1 + 1 + 1 = 10$ aristas examinadas. En el mismo grafo, la versión ingenua examinó 28 en las rondas y la de parada temprana 21. El resultado es el mismo: $d = (0, 3, 5, 6, 8)$ y $\pi = (\mathit{NIL}, 2, 0, 1, 3)$.

Ojo con la comparación

Son las cifras de este grafo. En el peor caso las tres versiones cuestan $O(VE)$.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

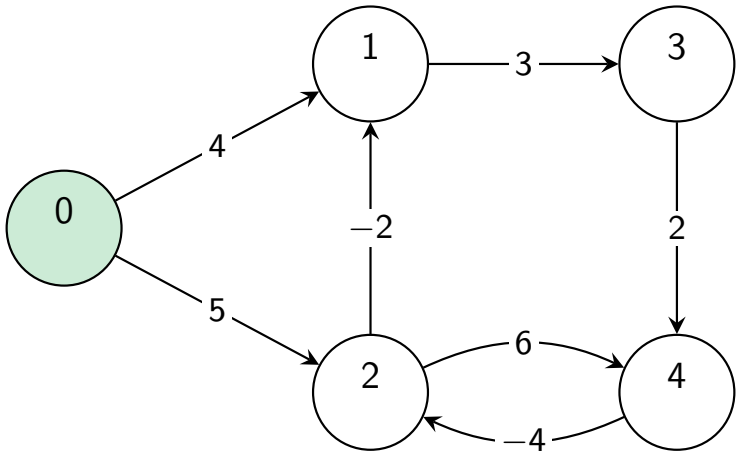
9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Una sola arista cambiada



El mismo grafo, con $w(4,2) = -4$

El ciclo $\langle 2, 1, 3, 4, 2\rangle$ pasa a pesar $-2 + 3 + 2 - 4 = -1$ y es alcanzable desde 0. Entonces $\delta(0,1) = \delta(0,2) = \delta(0,3) = \delta(0,4) = -\infty$.

Pregunta

Las cuatro rondas de Bellman-Ford van a terminar igual, con números finitos en d. ¿Cómo se entera el algoritmo de que esos números no sirven?

Bellman-Ford con el ciclo

Al final de	d					π				
	0	1	2	3	4	0	1	2	3	4
inicio	0	∞	∞	∞	∞	–	–	–	–	–
ronda 1	0	3	5	7	9	–	2	0	1	3
ronda 2	0	3	4	6	8	–	2	4	1	3
ronda 3	0	2	4	6	8	–	2	4	1	3
ronda 4	0	2	3	5	7	–	2	4	1	3

Respuesta: la revisión final

Con $d = (0, 2, 3, 5, 7)$, la arista $(2, 1)$ está tensa:
$1.d = 2 > 2.d + w(2, 1) = 3 - 2 = 1$. La función devuelve `False`. Una quinta ronda bajaría algo más: cada vuelta al ciclo mejora un poco.

El ciclo está en π

Desde 1: $1.\pi = 2$, $2.\pi = 4$, $4.\pi = 3$, $3.\pi = 1$. Subir por π desde la arista tensa termina dando vueltas en el ciclo.

Moore con el ciclo

Las siete primeras salidas son las de la traza sin ciclo. Desde la octava, cuando sale 4 con $4.d = 8$, la arista $(4, 2)$ ya baja a 2.

Sale	Cambia	d					Cola	veces				
		0	1	2	3	4		0	1	2	3	4
4	2	0	3	4	6	8	2	1	2	2	2	2
2	1	0	2	4	6	8	1	1	3	2	2	2
1	3	0	2	4	5	8	3	1	3	2	3	2
3	4	0	2	4	5	7	4	1	3	2	3	3
4	2	0	2	3	5	7	2	1	3	3	3	3
2	1	0	1	3	5	7	1	1	4	3	3	3
1	3	0	1	3	4	7	3	1	4	3	4	3
3	4	0	1	3	4	6	4	1	4	3	4	4
4	2	0	1	2	4	6	2	1	4	4	4	4
2	1	0	0	2	4	6	1	1	5	4	4	4

Moore con el ciclo

Lo que se ve

La cola nunca se vacía: cada vuelta al ciclo baja 1 en los cuatro vértices y los vuelve a meter. En la salida 17, 1 entra por quinta vez; `veces[1]` $= 5 = |V|$ y la función devuelve `False` sin esperar más.

Sin el contador

Sin `veces`, el `while` con un ciclo negativo alcanzable no termina.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

UVa 336, A Node Too Far

El problema

Una red no dirigida de a lo sumo 30 nodos con identificadores arbitrarios. Para cada consulta, un nodo inicial y un TTL: cuántos nodos *no* recibe un mensaje que avanza a lo sumo TTL saltos.

Enunciado: `https://onlinejudge.org/external/3/336.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=272`

Cómo atacarlo

Es un camino más corto sin pesos: una amplitud desde el nodo inicial y se cuentan los nodos con $d = -1$ o $d > TTL$. Los identificadores no son $0..n-1$: un diccionario les asigna un índice. La trampa: la numeración de los casos sigue de una red a la siguiente. Conviene prever también una consulta cuyo nodo no aparezca en ninguna conexión.

UVa 558, Wormholes

El problema

Hasta 1000 sistemas y 2000 agujeros de gusano dirigidos, cada uno con un salto en el tiempo t, con $-1000 \leq t \leq 1000$. Desde el sistema 0 se llega a todos. ¿Hay un ciclo que lleve indefinidamente al pasado?

Enunciado: `https://onlinejudge.org/external/5/558.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=499`

Cómo atacarlo

Un ciclo al pasado es un ciclo de peso negativo, y todos son alcanzables desde 0. Se corre Bellman-Ford desde 0 y la respuesta sale de la revisión final: si queda una arista tensa, se imprime `possible`. Con $V \cdot E = 2 \times 10^6$ alcanza la versión por rondas; la de Moore también sirve, con el contador `veces`. La trampa: no confundir "hay aristas negativas" con "hay ciclo negativo".

UVa 11280, Flying to Fredericton

El problema

Hasta 100 ciudades y 1000 vuelos con costo, puede haber varios entre las mismas dos ciudades. Para cada consulta, un máximo de escalas: el vuelo más barato de Calgary a Fredericton con a lo sumo esas escalas.

Enunciado: `https://onlinejudge.org/external/112/11280.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2255`

Cómo atacarlo

k escalas son a lo sumo $k+1$ vuelos, y la ronda i de Bellman-Ford es la que alcanza los caminos de i aristas: se hacen $k+1$ rondas y se lee d de Fredericton. La trampa: dentro de una ronda, relajar con los d que la misma ronda acaba de cambiar encadena dos vuelos en una sola ronda. Cada ronda tiene que leer de una copia de d tomada al empezar. Los nombres de las ciudades se pasan a índices con un diccionario.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Errores comunes

Usar la amplitud con pesos

La amplitud minimiza el número de aristas, no el peso. Solo sirve cuando todas las aristas pesan lo mismo.

Una ronda de menos

Con `ronda = 1` y `while ronda < n - 1` se hacen $|V| - 2$ rondas. Un camino más corto de $|V| - 1$ aristas que el orden de las listas recorre al revés queda sin terminar, y la revisión lo toma por un ciclo negativo.

Relajar desde un ∞

En lenguajes con enteros, ∞ suele ser un número grande, y $\infty + w$ con $w < 0$ da un valor menor que ∞: un vértice no alcanzado termina con una distancia. Se relaja solo si $u.d$ es finito. En Python, `float('inf')` ya cumple $\infty + w = \infty$.

Errores comunes

Leer d cuando hay un ciclo negativo

Si la función devuelve `False`, los números de d no son distancias. Hay que mirar primero el resultado de la revisión.

Moore sin `en_cola` o sin `veces`

Sin `en_cola` la cola se llena de copias. Sin `veces`, un ciclo negativo deja el ciclo `while` corriendo para siempre.

Creer que un ciclo negativo lejano da *FALSE*

Un ciclo negativo al que no se llega desde s no cambia ninguna distancia desde s, y Bellman-Ford devuelve *TRUE*. Para detectar cualquier ciclo negativo del grafo se agrega un vértice nuevo con aristas de peso 0 hacia todos y se corre desde él.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Ejercicios

1. La amplitud a mano

Sea A el grafo no dirigido con $V = \{0, \ldots, 6\}$ y aristas 0–3, 0–4, 3–1, 4–1, 4–5, 1–6, 5–6, 5–2, con las listas en orden creciente. Ejecute la amplitud desde 0: escriba el orden en que salen de la cola y d y π de cada vértice, y reconstruya el camino a 6 y el camino a 2.

2. Pesos enteros con la amplitud

En un grafo dirigido con pesos enteros positivos, cada arista (u, v) de peso w se cambia por un camino de w aristas sin peso a través de $w - 1$ vértices nuevos. Muestre que la amplitud sobre el grafo nuevo da δ del grafo original, y diga cuánto cuesta en función de V, E y el peso máximo W. ¿Cuándo no conviene?

Ejercicios

3. Bellman-Ford a mano

Sea B el grafo dirigido con $V = \{0, \ldots, 4\}$, fuente 0 y estas listas de adyacencia, en este orden: 0: $(4, 6)$, $(3, 4)$, $(2, 7)$; 1: $(4, 5)$; 2: $(1, -1)$; 3: $(2, 2)$; 4: $(3, -3)$, donde (v, w) es una arista hacia v de peso w. Llene la tabla de la traza ronda por ronda, como lo hace el código, diga en qué ronda para `bellman_ford` y cuántas aristas examinó.

4. El orden de las aristas

Si las aristas de B se guardan en una sola lista, como en CLRS, dé un orden de esa lista con el que una sola ronda deje todas las distancias correctas. Explique por qué con la propiedad de relajación de caminos. ¿Existe un orden así para cualquier grafo sin ciclos negativos?

Ejercicios

5. Moore a mano

Ejecute `bellman_ford_moore` sobre B con las listas de cada vértice en el orden del ejercicio 3. Escriba la tabla con lo que sale, lo que cambia, d, la cola y `veces`, y cuente las aristas examinadas.

6. El ciclo negativo y π

Sea C el grafo dirigido con $V = \{0, \ldots, 5\}$, fuente 0 y aristas $(0, 1)$ de peso 2, $(1, 2)$ de peso 1, $(2, 3)$ de peso -4, $(3, 1)$ de peso 2, $(3, 4)$ de peso 3, $(5, 4)$ de peso 1. Diga $\delta(0, v)$ para cada v. Ejecute Bellman-Ford, señale una arista tensa en la revisión y recupere el ciclo subiendo por π. ¿Por qué hay que subir $|V|$ veces antes de empezar a anotar?

Ejercicios

7. La parada temprana es correcta

Demuestre que si en una ronda de `bellman_ford` no cambia ningún d, entonces no hay ciclos negativos alcanzables desde s y $v.d = \delta(s, v)$ para todo v. Escriba la demostración en cuatro partes: teorema, estrategia, desarrollo y conclusión, y diga qué propiedad de 24.5 usa en cada paso.

8. A lo sumo k aristas

Escriba una función que reciba G, s y k y devuelva, para cada v, el peso del camino más barato de s a v con a lo sumo k aristas. Dé un grafo de tres vértices en el que la versión que no copia d al empezar cada ronda da una respuesta equivocada para $k = 1$.

Pistas

Ejercicios 1 a 4

1. El árbol de la amplitud tiene 6 aristas; 2 queda a tres aristas de 0.
2. El grafo nuevo tiene hasta $V + E(W - 1)$ vértices. Compare ese tamaño con VE.
3. El camino más corto a 1 es $\langle 0, 4, 3, 2, 1 \rangle$ y recorre los vértices en orden decreciente, el contrario al del código. Las distancias finales son $(0, 4, 5, 3, 6)$.
4. Ordene las aristas para que las de cada camino más corto aparezcan en el orden del camino. La respuesta a la pregunta general depende de que el árbol de caminos más cortos exista.

Pistas

Ejercicios 5 a 8

5. Moore examina 7 aristas; las rondas, 28. Explique de dónde sale tanta diferencia en este grafo.
6. El ciclo es $\langle 1, 2, 3, 1 \rangle$, de peso -1. 5 no se alcanza desde 0. Un vértice tenso puede colgar del ciclo sin estar en él.
7. Por contradicción, como en el Teorema 24.4: si no hay aristas tensas, la suma por un ciclo no puede ser negativa.
8. El ejercicio no tiene pista: es el de UVa 11280 en pequeño.

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Lo que sigue

Grafos dirigidos sin ciclos

Bellman-Ford hace rondas porque no sabe en qué orden relajar. En un grafo dirigido acíclico ese orden existe y ya se sabe calcular.

Para investigar

- CLRS, Sección 24.2: si las aristas se relajan una sola vez, vértice por vértice en orden topológico, ¿por qué basta una pasada? ¿Qué propiedad de 24.5 lo garantiza, y cuánto cuesta?
- CLRS, Sección 24.3, el algoritmo de Dijkstra: si ningún peso es negativo, ¿qué vértice conviene sacar primero, y por qué una cola de prioridad reemplaza a la cola de Moore?

Contenido

1 Antes de empezar

2 Recuento: la búsqueda en amplitud

3 Caminos más cortos con pesos

4 Relajación

5 Bellman-Ford

6 La variante de Moore

7 Cuando hay un ciclo negativo

8 Problemas de juez

9 Errores comunes

10 Ejercicios

11 Lo que sigue

12 Referencias

Referencias



Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Sección 22.2, Teorema 22.5 (amplitud y caminos más cortos sin pesos); Capítulo 24, introducción, Lema 24.1 y relajación; Sección 24.1, Lema 24.2 y Teorema 24.4 (Bellman-Ford); Sección 24.5, Lemas 24.10, 24.11, 24.14 y 24.15 y Corolario 24.12 (propiedades de la relajación).



Moore, E. F. The shortest path through a maze. En *Proceedings of the International Symposium on the Theory of Switching*, Harvard University Press, 1959, pp. 285–292.



Halim, S., Halim, F., Effendy, S. *Competitive Programming 4*. Lulu, 2020. Capítulo 4: caminos más cortos desde una fuente, Bellman-Ford y su variante con cola.



Erickson, J. *Algorithms*. 2019. Capítulo 8, caminos más cortos: relajación y Bellman-Ford.